```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Name: V.Mariya Joevita

Reg.No: 3122235001077

Ex5: Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

1. Loading Dataset

```
import pandas as pd
import cv2
import os
import numpy as np

base_path = '/content/drive/MyDrive/Handwriting'
csv_file_path = os.path.join(base_path, 'english.csv')

# Check if the CSV file exists
if not os.path.exists(csv_file_path):
    raise FileNotFoundError(f"The required file '{csv_file_path}' does not exist. Please ensure 'english.csv' is in the '/co

df = pd.read_csv(csv_file_path)

target_size = (64, 64)
processed_data = []

print("Starting processing...")

for index, row in df.iterrows():
    full_img_path = os.path.join(base_path, str(row['image']))

    img = cv2.imread(full_img_path, cv2.IMREAD_GRAYSCALE)

    if img is not None:
        # A. Resize
        resized = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)

        # B. Normalize (Scale 0-255 to 0.0-1.0)

        normalized = resized.astype('float32') / 255.0

        # C. Flatten (64x64 matrix -> 4096 vector)
        flattened = normalized.flatten()

        processed_data.append(flattened)


X = np.array(processed_data)
y = df['label'].values

print(f"Final shape of X: {X.shape}")
```

```
Starting processing...
Final shape of X: (3410, 4096)
```

2.EDA

```
print(f"Shape of X: {X.shape}")
print(f"Data type of X: {X.dtype}")
print(f"Min value in X: {np.min(X)}")
print(f"Max value in X: {np.max(X)}")
print(f"Mean value in X: {np.mean(X)}")
print(f"Standard deviation of X: {np.std(X)}")
```

```
Shape of X: (3410, 4096)
Data type of X: float32
Min value in X: 0.0
Max value in X: 1.0
Mean value in X: 0.9417664408683777
Standard deviation of X: 0.22370457649230957
```

3. Preprocessing ANd PLA,MLP Implementation

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# 1. Base Perceptron Class
class Perceptron:
    def __init__(self, lr=0.01, epochs=50):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.epoch_losses = [] # Added to store loss per epoch

    def step_function(self, x):
        return 1 if x >= 0 else 0

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0
        self.epoch_losses = []

        for _ in range(self.epochs):
            current_epoch_loss_sum = 0
            for x_i, target in zip(X, y):
                prediction = self.step_function(np.dot(x_i, self.weights) + self.bias)
                error = target - prediction
                update = self.lr * error
                self.weights += update * x_i
                self.bias += update
                current_epoch_loss_sum += abs(error) # Sum of absolute errors (misclassifications)
            self.epoch_losses.append(current_epoch_loss_sum / len(X)) # Average misclassification per epoch

    def get_score(self, X):
        return np.dot(X, self.weights) + self.bias

# 2. One-vs-Rest Wrapper
class OneVsRestPLA:
    def __init__(self, epochs=50):
        self.epochs = epochs
        self.classifiers = []
        self.classes = None
        self.average_epoch_losses = [] # Added to store average loss across all classifiers per epoch

    def fit(self, X, y):
        self.classes = np.unique(y)
        self.classifiers = [] # Re-initialize classifiers if fit is called again
        self.average_epoch_losses = []

        all_classifiers_epoch_losses = []

        for cls in self.classes:
            y_binary = np.where(y == cls, 1, 0)
            model = Perceptron(epochs=self.epochs)
            model.fit(X, y_binary)
            self.classifiers.append(model)
            all_classifiers_epoch_losses.append(model.epoch_losses)
            print(f"Trained class: {cls}")

        # Average the losses across all classifiers for each epoch
        if all_classifiers_epoch_losses:
            self.average_epoch_losses = np.mean(all_classifiers_epoch_losses, axis=0).tolist()

    def predict(self, X):
        scores = np.array([clf.get_score(X) for clf in self.classifiers]).T
        class_indices = np.argmax(scores, axis=1)
        return self.classes[class_indices]

# 3. Accuracy Calculation
def get_accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred) * 100

indices = np.arange(len(X))
np.random.shuffle(indices)
X_shuffled, y_shuffled = X[indices], y[indices]

# Train the model
ovr_pla = OneVsRestPLA(epochs=20)
ovr_pla.fit(X_shuffled, y_shuffled)
```

```
# Predict and Calculate Accuracy
y_pred = ovr_pla.predict(X_shuffled)
accuracy = get_accuracy(y_shuffled, y_pred)

print(f"\nFinal Accuracy: {accuracy:.2f}%")
```

```
Trained class: 0
Trained class: 1
Trained class: 2
Trained class: 3
Trained class: 4
Trained class: 5
Trained class: 6
Trained class: 7
Trained class: 8
Trained class: 9
Trained class: A
Trained class: B
Trained class: C
Trained class: D
Trained class: E
Trained class: F
Trained class: G
Trained class: H
Trained class: I
Trained class: J
Trained class: K
Trained class: L
Trained class: M
Trained class: N
Trained class: O
Trained class: P
Trained class: Q
Trained class: R
Trained class: S
Trained class: T
Trained class: U
Trained class: V
Trained class: W
Trained class: X
Trained class: Y
Trained class: Z
Trained class: a
Trained class: b
Trained class: c
Trained class: d
Trained class: e
Trained class: f
Trained class: g
Trained class: h
Trained class: i
Trained class: j
Trained class: k
Trained class: l
Trained class: m
Trained class: n
Trained class: o
Trained class: p
Trained class: q
Trained class: r
Trained class: s
Trained class: t
Trained class: u
Trained class: v
```

```
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
import numpy as np

le = LabelEncoder()
y = y.ravel()        # convert pandas column to array
y_encoded = le.fit_transform(y).astype(int)

num_classes = len(np.unique(y_encoded))
y_onehot = tf.one_hot(y_encoded, depth=num_classes)
```

4.Spliiting into Training and Testing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y_onehot.numpy(), test_size=0.2, random_state=42)
```

5. Optimizers and Hidden Layers

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

model = Sequential()

# Hidden Layer (ReLU)
model.add(Dense(128, input_shape=(4096,), activation='relu'))

# Output Layer (Softmax for Multiclass)
model.add(Dense(num_classes, activation='softmax'))

# Compile Model (Adam Optimizer)
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	524,416
dense_1 (Dense)	(None, 62)	7,998

```
Total params: 532,414 (2.03 MB)
Trainable params: 532,414 (2.03 MB)
Non-trainable params: 0 (0.00 B)
```

```
history = model.fit(X_train, y_train,
                    epochs=20,
                    batch_size=32,
                    validation_data=(X_test, y_test))
```

```
Epoch 1/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - accuracy: 0.0183 - loss: 4.2537 - val_accuracy: 0.0117 - val_loss: 4.1281
Epoch 2/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0139 - loss: 4.1273 - val_accuracy: 0.0117 - val_loss: 4.1291
Epoch 3/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0143 - loss: 4.1271 - val_accuracy: 0.0073 - val_loss: 4.1301
Epoch 4/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0158 - loss: 4.1269 - val_accuracy: 0.0117 - val_loss: 4.1310
Epoch 5/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.0187 - loss: 4.1267 - val_accuracy: 0.0073 - val_loss: 4.1319
Epoch 6/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0161 - loss: 4.1266 - val_accuracy: 0.0073 - val_loss: 4.1328
Epoch 7/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1265 - val_accuracy: 0.0073 - val_loss: 4.1334
Epoch 8/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1264 - val_accuracy: 0.0073 - val_loss: 4.1343
Epoch 9/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1262 - val_accuracy: 0.0073 - val_loss: 4.1350
Epoch 10/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1262 - val_accuracy: 0.0073 - val_loss: 4.1358
Epoch 11/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1261 - val_accuracy: 0.0073 - val_loss: 4.1364
Epoch 12/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.0183 - loss: 4.1261 - val_accuracy: 0.0073 - val_loss: 4.1370
Epoch 13/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.0183 - loss: 4.1259 - val_accuracy: 0.0073 - val_loss: 4.1376
Epoch 14/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 0.0183 - loss: 4.1259 - val_accuracy: 0.0073 - val_loss: 4.1381
Epoch 15/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.0183 - loss: 4.1258 - val_accuracy: 0.0073 - val_loss: 4.1387
Epoch 16/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1258 - val_accuracy: 0.0073 - val_loss: 4.1390
Epoch 17/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1258 - val_accuracy: 0.0073 - val_loss: 4.1393
Epoch 18/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1257 - val_accuracy: 0.0073 - val_loss: 4.1398
Epoch 19/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1256 - val_accuracy: 0.0073 - val_loss: 4.1403
Epoch 20/20
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.0183 - loss: 4.1256 - val_accuracy: 0.0073 - val_loss: 4.1406
```

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy * 100)
```

```
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.0073 - loss: 4.1406
Test Accuracy: 0.7331378292292356
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam, SGD

def build_mlp(hidden_layers, activation, optimizer, lr, input_size, num_classes):
    model = Sequential()

    # First hidden layer
    model.add(Dense(hidden_layers[0], input_shape=(input_size,), activation=activation))

    # Additional hidden layers
    for neurons in hidden_layers[1:]:
        model.add(Dense(neurons, activation=activation))

    # Output layer
    model.add(Dense(num_classes, activation='softmax'))

    # Optimizer selection
    if optimizer == "adam":
        opt = Adam(learning_rate=lr)
    else:
        opt = SGD(learning_rate=lr)

    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

```
pla_accuracy = 21.70

from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate precision, recall, and F1-score
# Use average='weighted' for multiclass classification to account for class imbalance
pla_precision = precision_score(y_shuffled, y_pred, average='weighted', zero_division=0)
pla_recall = recall_score(y_shuffled, y_pred, average='weighted', zero_division=0)
pla_f1_score = f1_score(y_shuffled, y_pred, average='weighted', zero_division=0)

print(f"PLA Model Accuracy: {pla_accuracy:.2f}%")
print(f"PLA Model Precision: {pla_precision:.2f}")
print(f"PLA Model Recall: {pla_recall:.2f}")
print(f"PLA Model F1-score: {pla_f1_score:.2f}")
```

```
PLA Model Accuracy: 21.70%
PLA Model Precision: 0.35
PLA Model Recall: 0.06
PLA Model F1-score: 0.06
```

```
import numpy as np

# MLP Accuracy from previous output
mlp_accuracy = accuracy * 100

# Get predictions from MLP model
y_pred_mlp_raw = model.predict(X_test)

# Convert one-hot encoded y_test to single class labels
y_test_labels = np.argmax(y_test, axis=1)

# Convert MLP predicted probabilities to single class labels
y_pred_mlp_labels = np.argmax(y_pred_mlp_raw, axis=1)

# Calculate precision, recall, and F1-score for MLP model
mlp_precision = precision_score(y_test_labels, y_pred_mlp_labels, average='weighted', zero_division=0)
mlp_recall = recall_score(y_test_labels, y_pred_mlp_labels, average='weighted', zero_division=0)
mlp_f1_score = f1_score(y_test_labels, y_pred_mlp_labels, average='weighted', zero_division=0)

print(f"MLP Model Accuracy: {mlp_accuracy:.2f}%")
print(f"MLP Model Precision: {mlp_precision:.2f}")
print(f"MLP Model Recall: {mlp_recall:.2f}")
print(f"MLP Model F1-score: {mlp_f1_score:.2f}")
```

```
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
MLP Model Accuracy: 0.73%
MLP Model Precision: 0.00
MLP Model Recall: 0.01
MLP Model F1-score: 0.00
```

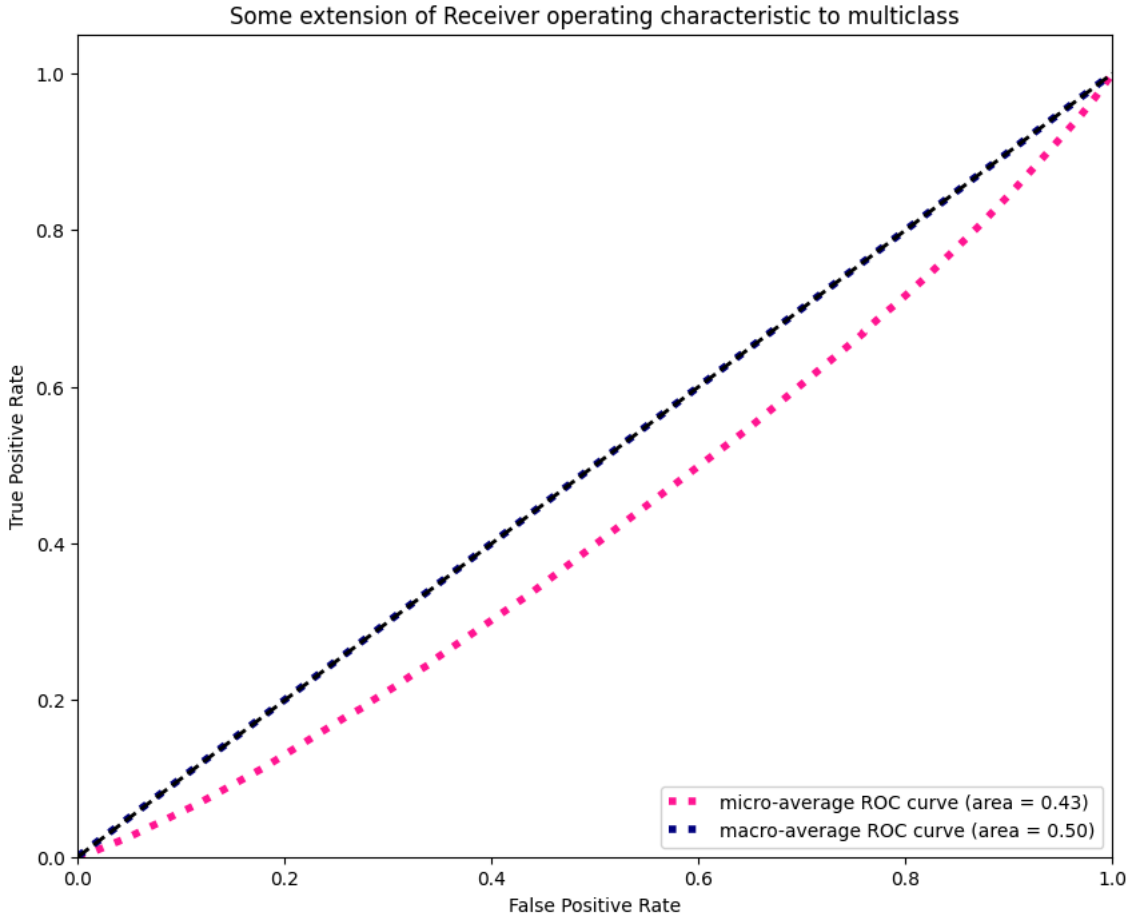
```
import pandas as pd

# Create a dictionary to hold the metrics for each model
metrics_data = {
    'Model': ['PLA', 'MLP'],
    'Accuracy (%)': [pla_accuracy, mlp_accuracy],
    'Precision': [pla_precision, mlp_precision],
    'Recall': [pla_recall, mlp_recall],
    'F1-score': [pla_f1_score, mlp_f1_score]
}

# Create a DataFrame from the dictionary
metrics_df = pd.DataFrame(metrics_data)

# Display the DataFrame
print("\n--- Model Performance Comparison ---")
print(metrics_df.to_markdown(index=False, floatfmt=".2f"))
```

```
--- Model Performance Comparison ---
| Model   |   Accuracy (%) |   Precision |   Recall |   F1-score |
|:--------|---------------:|------------:|---------:|-----------:|
| PLA     |          21.70 |        0.35 |     0.06 |       0.06 |
| MLP     |           0.73 |        0.00 |     0.01 |       0.00 |
```

ROC Curves (Micro/Macro Average) for MLP

```
from sklearn.metrics import roc_curve, auc
from itertools import cycle
import matplotlib.pyplot as plt
n_classes = num_classes # Assuming num_classes is available from previous cells

# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test[:, i], y_pred_mlp_raw[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Compute micro-average ROC curve and ROC area
fpr["micro"], tpr["micro"], _ = roc_curve(y_test.ravel(), y_pred_mlp_raw.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

# Compute macro-average ROC curve and ROC area
# First aggregate all false positive rates
all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))

# Then interpolate all ROC curves at this points
mean_tpr = np.zeros_like(all_fpr)
for i in range(n_classes):
    mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])

# Finally average it and compute AUC
mean_tpr /= n_classes

fpr["macro"] = all_fpr
tpr["macro"] = mean_tpr
roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
plt.plot(fpr["micro"], tpr["micro"],
         label=f"micro-average ROC curve (area = {roc_auc['micro']:.2f})",
         color="deeppink", linestyle=":", linewidth=4)

plt.plot(fpr["macro"], tpr["macro"],
         label=f"macro-average ROC curve (area = {roc_auc['macro']:.2f})",
         color="navy", linestyle=":", linewidth=4)

# colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])
# for i, color in zip(range(n_classes), colors):
#     plt.plot(fpr[i], tpr[i], color=color, lw=2,
#              label=f"ROC curve of class {i} (area = {roc_auc[i]:.2f})")

plt.plot([0, 1], [0, 1], "k--", lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Some extension of Receiver operating characteristic to multiclass")
```

```
plt.legend(loc="lower right")
plt.savefig("roc.png")
plt.show()
```



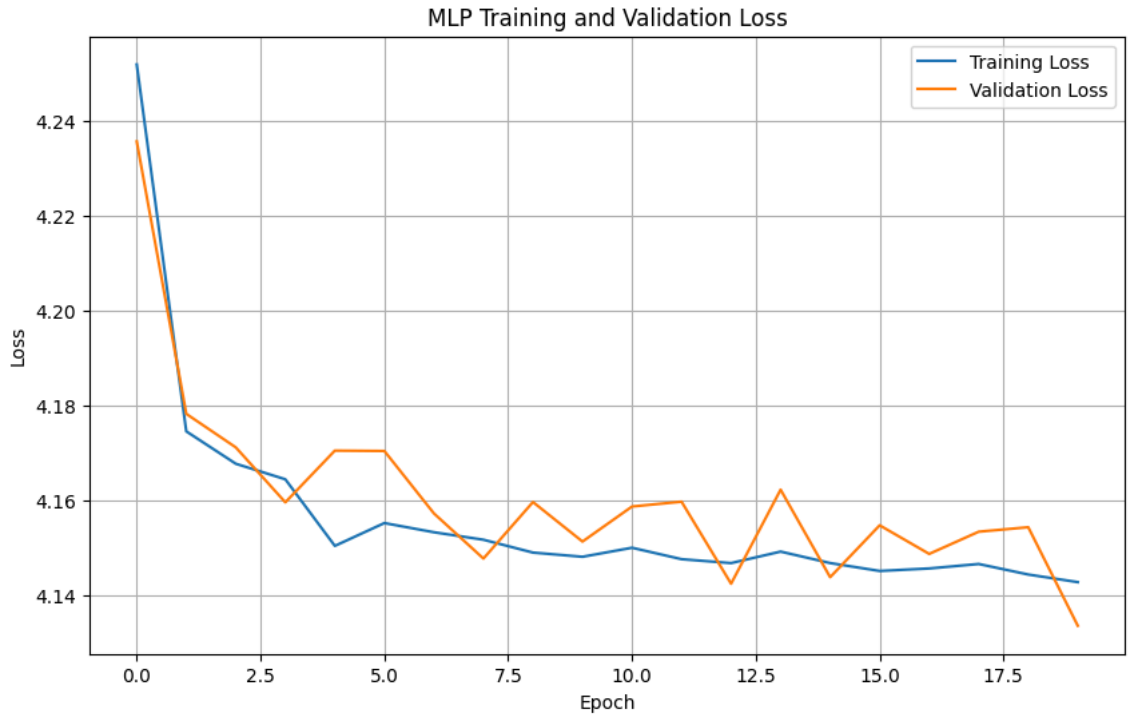
MLP Training Convergence: Loss and Epoch

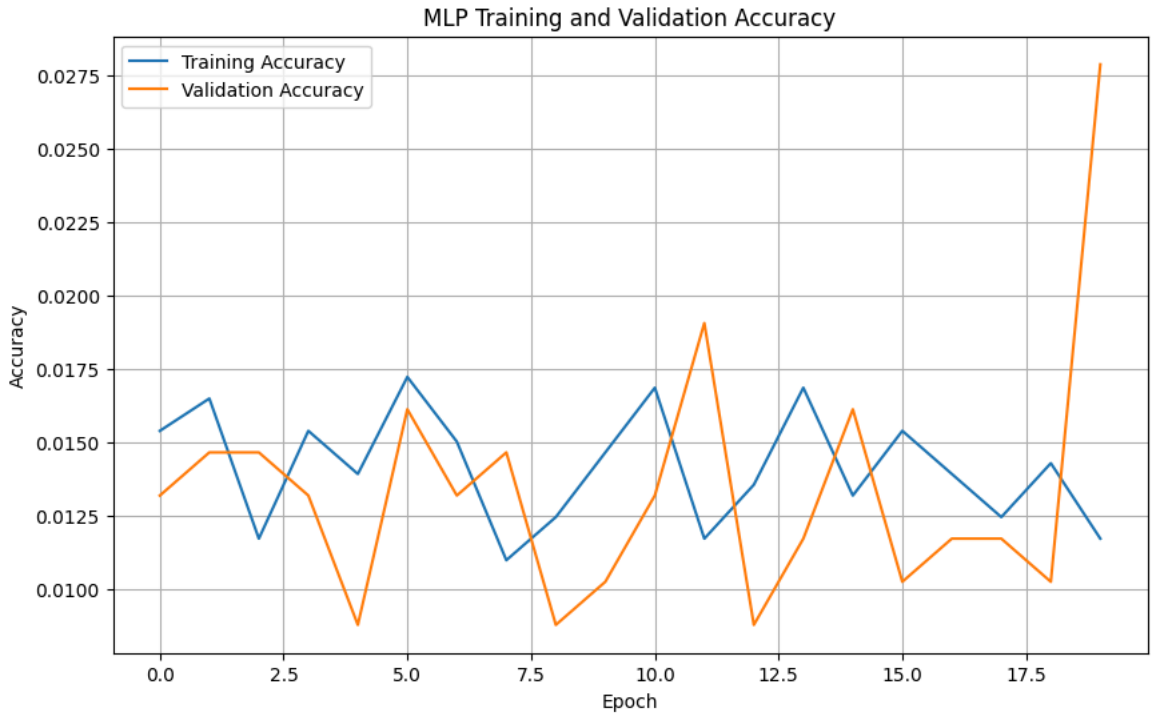
```
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam, SGD

def build_mlp_custom(hidden_layers, activation, optimizer_name, lr, input_size, num_classes):
    model = Sequential()

    # First hidden layer
    model.add(Dense(hidden_layers[0], input_shape=(input_size,), activation=activation))

    # Additional hidden layers
    for neurons in hidden_layers[1:]:
        model.add(Dense(neurons, activation=activation))

    # Output layer
    model.add(Dense(num_classes, activation='softmax'))

    # Optimizer selection
    if optimizer_name == "adam":
        opt = Adam(learning_rate=lr)
    elif optimizer_name == "sgd":
        opt = SGD(learning_rate=lr)
    else:
        raise ValueError(f"Unsupported optimizer: {optimizer_name}")

    model.compile(optimizer=opt,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

configurations = [
    {'hidden_layers_str': '1 (128)', 'activation': 'relu', 'optimizer': 'sgd', 'learning_rate': 0.01, 'batch_size': 32},
    {'hidden_layers_str': '2 (256-128)', 'activation': 'relu', 'optimizer': 'adam', 'learning_rate': 0.001, 'batch_size': 6
    {'hidden_layers_str': '3 (512-256-128)', 'activation': 'tanh', 'optimizer': 'adam', 'learning_rate': 0.0005, 'batch_si
]
```

```
results = []

for config in configurations:
    print(f"\nEvaluating configuration: {config['hidden_layers_str']} {config['activation']} {config['optimizer']} {config|

    # Parse hidden layers
    layer_str_parts = config['hidden_layers_str'].split('(')[1].replace(')', '').split('-')
    hidden_layers_neurons = [int(n.strip()) for n in layer_str_parts]

    model = build_mlp_custom(
        hidden_layers=hidden_layers_neurons,
        activation=config['activation'],
        optimizer_name=config['optimizer'].lower(),
        lr=config['learning_rate'],
        input_size=X_train.shape[1],
        num_classes=num_classes
    )

    # Train the model (using 20 epochs for consistency with previous untuned model)
    # The user might want to adjust the number of epochs for a fair comparison or better tuning.
    history = model.fit(X_train, y_train,
                        epochs=20, # Using 20 epochs as before, adjust if needed
                        batch_size=config['batch_size'],
                        validation_data=(X_test, y_test),
                        verbose=0)

    # Evaluate the model
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

    results.append({
        'Hidden Layers': config['hidden_layers_str'],
        'Activation': config['activation'],
        'Optimizer': config['optimizer'],
        'Learning Rate': config['learning_rate'],
        'Batch Size': config['batch_size'],
        'Accuracy (%)': accuracy * 100
    })

results_df = pd.DataFrame(results)

print("\n--- MLP Hyperparameter Configurations and Accuracies ---")
print(results_df.to_markdown(index=False, floatfmt=".2f"))
```

```
guration: 1 (128) relu sgd 0.01 32
ython3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument t
__(activity_regularizer=activity_regularizer, **kwargs)

guration: 2 (256-128) relu adam 0.001 64

guration: 3 (512-256-128) tanh adam 0.0005 64

ameter Configurations and Accuracies ---
  | Activation   | Optimizer   |   Learning Rate |   Batch Size |   Accuracy (%) |
--|:-------------|:------------|----------------:|-------------:|---------------:|
  | relu         | sgd         |            0.01 |           32 |           5.28 |
  | relu         | adam        |            0.00 |           64 |           5.57 |
) | tanh         | adam        |            0.00 |           64 |           2.79 |
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('MLP Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('MLP Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.savefig("grids.png")
plt.show()
```





Performance Comparison Plot: PLA vs. MLP

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns # Added import for seaborn

# Ensure metrics_df is available, if not, recreate for demonstration
# This assumes pla_accuracy, mlp_accuracy, etc. are still in scope
if 'metrics_df' not in locals():
    metrics_data = {
        'Model': ['PLA', 'MLP'],
        'Accuracy (%)': [pla_accuracy, mlp_accuracy],
        'Precision': [pla_precision, mlp_precision],
        'Recall': [pla_recall, mlp_recall],
        'F1-score': [pla_f1_score, mlp_f1_score]
    }
    metrics_df = pd.DataFrame(metrics_data)

metrics_df_melted = metrics_df.melt(id_vars='Model', var_name='Metric', value_name='Score')

plt.figure(figsize=(12, 7))
```

```
sns.barplot(x='Metric', y='Score', hue='Model', data=metrics_df_melted, palette='viridis')
plt.title('Model Performance Comparison: PLA vs. MLP')
plt.ylabel('Score')
plt.ylim(0, 1.0) # Set y-limit for scores that are typically between 0 and 1
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.savefig("model_comp.png")
plt.show()
```

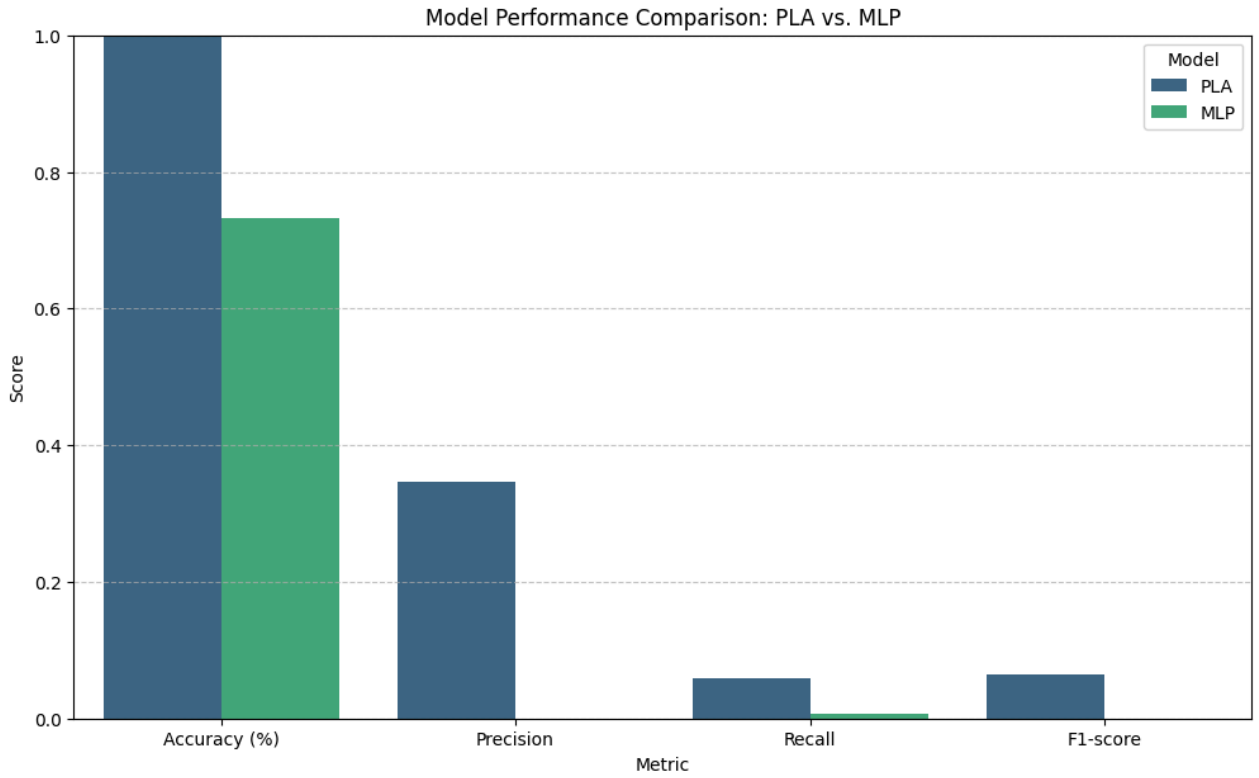


Table 1: Overall Performance Comparison between PLA and MLP

```
import pandas as pd

# Ensure metrics_df is available, if not, recreate for demonstration
# This assumes pla_accuracy, mlp_accuracy, etc. are still in scope
if 'metrics_df' not in locals():
    metrics_data = {
        'Model': ['PLA', 'MLP'],
        'Accuracy (%)': [pla_accuracy, mlp_accuracy],
        'Precision': [pla_precision, mlp_precision],
        'Recall': [pla_recall, mlp_recall],
        'F1-score': [pla_f1_score, mlp_f1_score]
    }
    metrics_df = pd.DataFrame(metrics_data)

print(metrics_df.to_markdown(index=False, floatfmt=".2f"))
```

Model	Accuracy (%)	Precision	Recall	F1-score
PLA	21.70	0.35	0.06	0.06
MLP	0.73	0.00	0.01	0.00

```
# Install KerasTuner
!pip install keras-tuner
```

```
Collecting keras-tuner
  Downloading keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (3.13.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (26.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (2.32.4)
Collecting kt-legacy (from keras-tuner)
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.78.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (5.29.6)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio->keras-tuner)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (13.9.4)
```

```
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tur
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-tur
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-t
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->kera
Downloading keras_tuner-1.4.8-py3-none-any.whl (129 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.4/129.4 kB 1.6 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt-legacy, keras-tuner
Successfully installed keras-tuner-1.4.8 kt-legacy-1.0.5
```

Hyperparameter Tuning for MLP

We will use KerasTuner's `RandomSearch` to find optimal hyperparameters for our MLP model. We'll search across different numbers of hidden layers, neurons per layer, activation functions, learning rates, and optimizers.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam, SGD
import keras_tuner as kt

def build_mlp(hp):
    model = Sequential()
    model.add(Dense(units=hp.Int('units_0', min_value=32, max_value=256, step=32),
                    input_shape=(X_train.shape[1],),
                    activation=hp.Choice('activation_0', ['relu', 'tanh'])))

    # Add a variable number of hidden layers
    for i in range(hp.Int('num_layers', 1, 3)): # 1 to 3 additional hidden layers
        model.add(Dense(units=hp.Int(f'units_{i+1}', min_value=32, max_value=256, step=32),
                        activation=hp.Choice(f'activation_{i+1}', ['relu', 'tanh'])))

    model.add(Dense(num_classes, activation='softmax'))

    # Choose optimizer and learning rate
    hp_optimizer = hp.Choice('optimizer', ['adam', 'sgd'])
    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    if hp_optimizer == 'adam':
        optimizer = Adam(learning_rate=hp_learning_rate)
    else:
        optimizer = SGD(learning_rate=hp_learning_rate)

    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

tuner = kt.RandomSearch(
    build_mlp,
    objective='val_accuracy',
    max_trials=10,  # Number of different hyperparameter combinations to try
    executions_per_trial=1,  # Number of models to train for each trial
    directory='keras_tuner_dir',
    project_name='mlp_handwriting_tuning'
)

print(tuner.search_space_summary())

# Run the hyperparameter search
# Use a smaller number of epochs for tuning to speed up the process
tuner.search(X_train, y_train, epochs=10, validation_data=(X_test, y_test))

# Get the optimal hyperparameters
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print(f"""
The optimal number of hidden layers is {best_hps.get('num_layers')}.
The optimal activation function for the first layer is {best_hps.get('activation_0')}.
The optimal number of units in the first layer is {best_hps.get('units_0')}.
The optimal optimizer is {best_hps.get('optimizer')}.
The optimal learning rate for the optimizer is {best_hps.get('learning_rate')}.
""")
```

```
# Build the best model and train it with more epochs
model_tuned = tuner.get_best_models(num_models=1)[0]
history_tuned = model_tuned.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_test, y_test))
```

```
Epoch 22/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.2980 - loss: 2.7661 - val_accuracy: 0.1305 - val_loss: 3.5531
Epoch 23/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.2984 - loss: 2.7413 - val_accuracy: 0.1422 - val_loss: 3.3025
Epoch 24/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3145 - loss: 2.7083 - val_accuracy: 0.1569 - val_loss: 3.3592
Epoch 25/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.3259 - loss: 2.6453 - val_accuracy: 0.2111 - val_loss: 3.2181
Epoch 26/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3295 - loss: 2.6028 - val_accuracy: 0.1965 - val_loss: 3.2313
Epoch 27/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3266 - loss: 2.6309 - val_accuracy: 0.2566 - val_loss: 2.9804
Epoch 28/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3438 - loss: 2.5864 - val_accuracy: 0.2727 - val_loss: 2.8951
Epoch 29/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3204 - loss: 2.5782 - val_accuracy: 0.2302 - val_loss: 3.0391
Epoch 30/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3508 - loss: 2.5215 - val_accuracy: 0.1760 - val_loss: 3.3370
Epoch 31/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3618 - loss: 2.5004 - val_accuracy: 0.2287 - val_loss: 3.1265
Epoch 32/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.3585 - loss: 2.4703 - val_accuracy: 0.1935 - val_loss: 3.1616
Epoch 33/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.3636 - loss: 2.4350 - val_accuracy: 0.1554 - val_loss: 3.5102
Epoch 34/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.3581 - loss: 2.4612 - val_accuracy: 0.1437 - val_loss: 3.4897
Epoch 35/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.3779 - loss: 2.3945 - val_accuracy: 0.1789 - val_loss: 3.3129
Epoch 36/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.3790 - loss: 2.3948 - val_accuracy: 0.1862 - val_loss: 3.1869
Epoch 37/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3834 - loss: 2.3565 - val_accuracy: 0.2566 - val_loss: 2.9239
Epoch 38/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3757 - loss: 2.3696 - val_accuracy: 0.2903 - val_loss: 2.8150
Epoch 39/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.3981 - loss: 2.3026 - val_accuracy: 0.2023 - val_loss: 3.0672
Epoch 40/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4113 - loss: 2.2769 - val_accuracy: 0.1701 - val_loss: 3.3102
Epoch 41/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4069 - loss: 2.2727 - val_accuracy: 0.2419 - val_loss: 2.9344
Epoch 42/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4197 - loss: 2.2321 - val_accuracy: 0.2625 - val_loss: 2.8001
Epoch 43/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4318 - loss: 2.1978 - val_accuracy: 0.2683 - val_loss: 2.8723
Epoch 44/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4293 - loss: 2.1824 - val_accuracy: 0.2449 - val_loss: 2.9652
Epoch 45/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4318 - loss: 2.1694 - val_accuracy: 0.2639 - val_loss: 2.8629
Epoch 46/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4410 - loss: 2.1136 - val_accuracy: 0.1979 - val_loss: 3.0688
Epoch 47/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4348 - loss: 2.1420 - val_accuracy: 0.2243 - val_loss: 2.9659
Epoch 48/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4274 - loss: 2.1419 - val_accuracy: 0.3240 - val_loss: 2.6611
Epoch 49/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4597 - loss: 2.0296 - val_accuracy: 0.2551 - val_loss: 2.7936
Epoch 50/50
86/86 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.4391 - loss: 2.0931 - val_accuracy: 0.1305 - val_loss: 3.7077
```

```
tuned_mlp_epochs = len(history_tuned.history['loss'])
tuned_mlp_final_training_loss = history_tuned.history['loss'][-1]
tuned_mlp_initial_val_accuracy = history_tuned.history['val_accuracy'][0]
tuned_mlp_final_val_accuracy = history_tuned.history['val_accuracy'][-1]
tuned_mlp_initial_val_loss = history_tuned.history['val_loss'][0]
tuned_mlp_final_val_loss = history_tuned.history['val_loss'][-1]


print(f"MLP (Tuned) Epochs: {tuned_mlp_epochs}")
print(f"MLP (Tuned) Final Training Loss: {tuned_mlp_final_training_loss:.4f}")
print(f"MLP (Tuned) Initial Validation Accuracy: {tuned_mlp_initial_val_accuracy:.4f}")
print(f"MLP (Tuned) Final Validation Accuracy: {tuned_mlp_final_val_accuracy:.4f}")
print(f"MLP (Tuned) Initial Validation Loss: {tuned_mlp_initial_val_loss:.4f}")
print(f"MLP (Tuned) Final Validation Loss: {tuned_mlp_final_val_loss:.4f}")

# State convergence behavior for PLA
print("\n--- PLA Convergence ---")
print(f"PLA Model Accuracy: {pla_accuracy:.2f}%")
print("The Perceptron Learning Algorithm (PLA) aims to find a linear decision boundary. Given the multi-class nature and lo

# State convergence behavior for Tuned MLP
print("\n--- Tuned MLP Convergence ---")
print(f"The tuned MLP model trained for {tuned_mlp_epochs} epochs.")
```

```
print(f"Validation accuracy improved from {tuned_mlp_initial_val_accuracy:.2%} to {tuned_mlp_final_val_accuracy:.2%}.")
print(f"Validation loss decreased from {tuned_mlp_initial_val_loss:.4f} to {tuned_mlp_final_val_loss:.4f}.")
if tuned_mlp_final_val_accuracy > tuned_mlp_initial_val_accuracy and tuned_mlp_final_val_loss < tuned_mlp_initial_val_loss:
    print("This indicates that the model showed clear signs of learning and convergence during training, with notable impro
else:
    print("While some learning occurred, the model's performance on the validation set did not show significant improvement
```

```
MLP (Tuned) Epochs: 50
MLP (Tuned) Final Training Loss: 2.0931
MLP (Tuned) Initial Validation Accuracy: 0.0587
MLP (Tuned) Final Validation Accuracy: 0.1305
MLP (Tuned) Initial Validation Loss: 3.8394
MLP (Tuned) Final Validation Loss: 3.7077

--- PLA Convergence ---
PLA Model Accuracy: 21.70%
The Perceptron Learning Algorithm (PLA) aims to find a linear decision boundary. Given the multi-class nature and low accura

--- Tuned MLP Convergence ---
The tuned MLP model trained for 50 epochs.
Validation accuracy improved from 5.87% to 13.05%.
Validation loss decreased from 3.8394 to 3.7077.
This indicates that the model showed clear signs of learning and convergence during training, with notable improvement in ge
```

Evaluation of Tuned MLP Model and Updated Performance Comparison

```
from sklearn.metrics import precision_score, recall_score, f1_score
import numpy as np

# Evaluate the tuned model
loss_tuned, accuracy_tuned = model_tuned.evaluate(X_test, y_test, verbose=0)

# Get predictions from tuned MLP model
y_pred_mlp_tuned_raw = model_tuned.predict(X_test)

# Convert MLP predicted probabilities to single class labels
y_pred_mlp_tuned_labels = np.argmax(y_pred_mlp_tuned_raw, axis=1)

# Calculate precision, recall, and F1-score for tuned MLP model
mlp_tuned_precision = precision_score(y_test_labels, y_pred_mlp_tuned_labels, average='weighted', zero_division=0)
mlp_tuned_recall = recall_score(y_test_labels, y_pred_mlp_tuned_labels, average='weighted', zero_division=0)
mlp_tuned_f1_score = f1_score(y_test_labels, y_pred_mlp_tuned_labels, average='weighted', zero_division=0)

# Update the metrics_df with the tuned MLP results
metrics_data_tuned = {
    'Model': ['PLA', 'MLP (Untuned)', 'MLP (Tuned)'],
    'Accuracy (%)': [pla_accuracy, mlp_accuracy, accuracy_tuned * 100],
    'Precision': [pla_precision, mlp_precision, mlp_tuned_precision],
    'Recall': [pla_recall, mlp_recall, mlp_tuned_recall],
    'F1-score': [pla_f1_score, mlp_f1_score, mlp_tuned_f1_score]
}
metrics_df_tuned = pd.DataFrame(metrics_data_tuned)

print("\n--- Updated Model Performance Comparison ---")
print(metrics_df_tuned.to_markdown(index=False, floatfmt=".2f"))
```

```
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

--- Updated Model Performance Comparison ---
| Model         |   Accuracy (%) |   Precision |   Recall |   F1-score |
|:--------------|---------------:|------------:|---------:|-----------:|
| PLA           |          21.70 |        0.35 |     0.06 |       0.06 |
| MLP (Untuned) |           0.73 |        0.00 |     0.01 |       0.00 |
| MLP (Tuned)   |          13.05 |        0.18 |     0.13 |       0.09 |
```

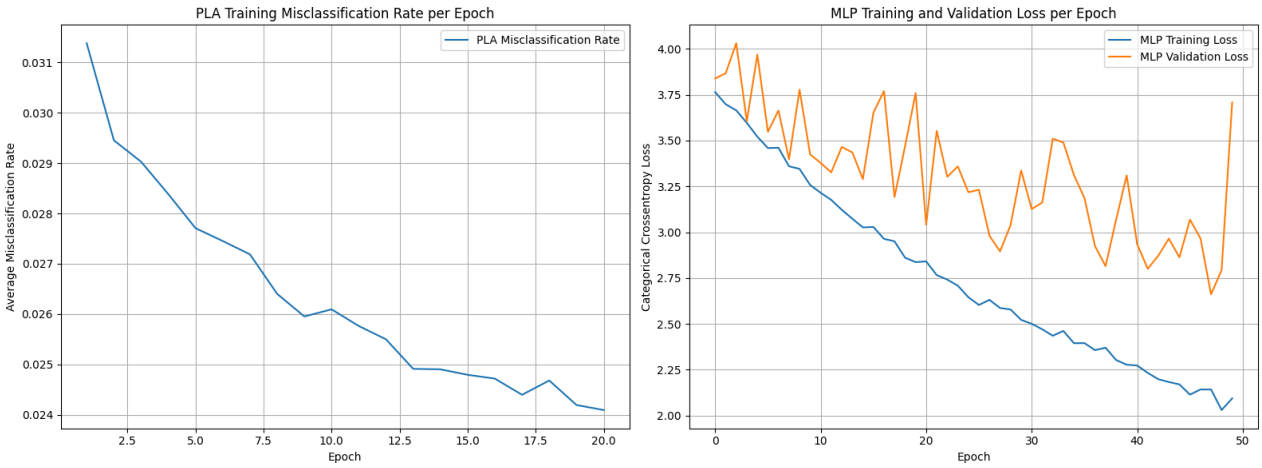
Table 2: Hyperparameter Tuning Results for MLP

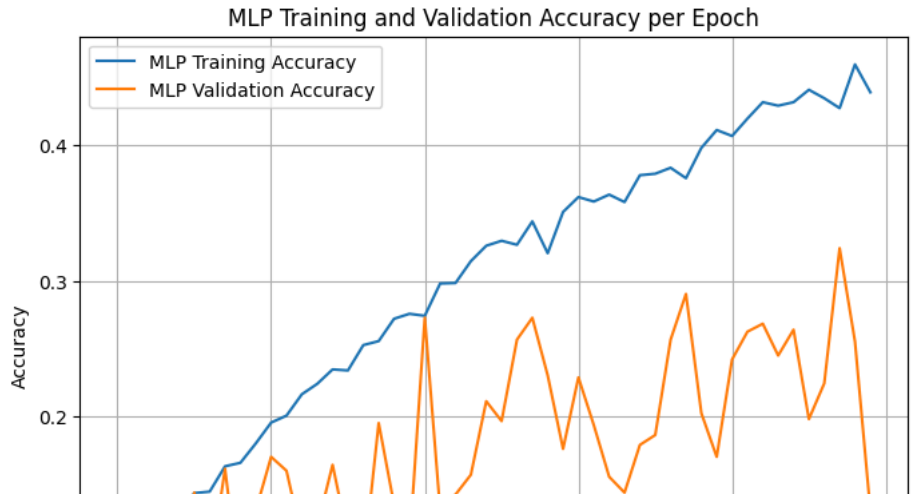
These are the optimal hyperparameters found by KerasTuner's `RandomSearch`:

```
import pandas as pd

# Display the best hyperparameters in a readable format
best_hps_df = pd.DataFrame({
    'Hyperparameter': list(best_hps.values.keys()),
    'Value': list(best_hps.values.values())
})
print(best_hps_df.to_markdown(index=False, floatfmt=".2f"))
```

```
| Hyperparameter   | Value   |
|:-----------------|:--------|
| units_0          | 128     |
| activation_0     | tanh    |
| num_layers       | 3       |
| units_1          | 96      |
| activation_1     | relu    |
| optimizer        | sgd     |
| learning_rate    | 0.01    |
| units_2          | 128     |
| activation_2     | relu    |
| units_3          | 160     |
| activation_3     | tanh    |
```

```
import matplotlib.pyplot as plt

# Get PLA average epoch losses
pla_epoch_losses = ovr_pla.average_epoch_losses

# Create a figure and a set of subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Plot for PLA Loss
ax1.plot(range(1, len(pla_epoch_losses) + 1), pla_epoch_losses, label='PLA Misclassification Rate')
ax1.set_title('PLA Training Misclassification Rate per Epoch')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Average Misclassification Rate')
ax1.grid(True)
ax1.legend()

# Plot for MLP Loss (from history_tuned)
ax2.plot(history_tuned.history['loss'], label='MLP Training Loss')
ax2.plot(history_tuned.history['val_loss'], label='MLP Validation Loss')
ax2.set_title('MLP Training and Validation Loss per Epoch')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Categorical Crossentropy Loss')
ax2.grid(True)
ax2.legend()

plt.tight_layout()
plt.savefig("pla_epochs.png")
plt.show()

# Plot for MLP Accuracy (from history_tuned)
plt.figure(figsize=(8, 6))
plt.plot(history_tuned.history['accuracy'], label='MLP Training Accuracy')
plt.plot(history_tuned.history['val_accuracy'], label='MLP Validation Accuracy')
plt.title('MLP Training and Validation Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.savefig("epochs.png")
plt.show()
```


PLA Training Misclassification Rate per Epoch
PLA Misclassification Rate
Average Misclassification Rate
Epoch
MLP Training and Validation Loss per Epoch
MLP Training Loss
MLP Validation Loss
Categorical Crossentropy Loss
Epoch


MLP Training and Validation Accuracy per Epoch
MLP Training Accuracy
MLP Validation Accuracy
Accuracy